Gloxad Backend Requirements: What the API Must Do Next

Oct 27, 2026 · @Samuel

Summary

The website is ready for the backend to build against. The backend must now deliver, in this order: unblock production (Phase 0), clean the live data, then the partner and institution modules, the trimester LMS, and live classes.

Done on the website as of 27 October:

- Every catalogue course is free, shown on the Free Courses page and the home page, with enrolment open.
- Invented ratings, learner counts, testimonials and other universities' courses are removed.
- The certificates page no longer crashes when the API returns an unexpected shape.
- Expired scholarships are hidden, and the tutor earnings page no longer shows sample bank and payout data.

Not yet done, and needs the backend or a decision: the trimester application path, the catalogue sign-in question, and real company and social links.

Phase 0: unblock production

These block uploads, production use and testing. Nothing else can be finished end to end until they are done.

Item	What the backend must do	Why it matters
Media bucket	Restore or create the bucket and confirm its name and region. Today the account has `amzn-v3-buccket-gloxad-sstone` in eu-north-1 and no `gloxad-bucket`. Re-seed the category, thumbnail and sample images.	Every image on the site returns 404 without it

Item	What the backend must do	Why it matters
Bucket CORS	Allow PUT, GET and HEAD from `http://localhost:3000`, `https://www.gloxadacademy.com` and `https://gloxadacademy.com`. Allow `Content-Type` and `x-amz-*` headers.	Browser uploads fail without it
Upload folders	Accept `avatars/`, `categories/`, `courses/`, `certifications/`, `logos/`, `signatures/`, `documents/`, `submissions/`, `certificates/`. Enforce type and size in the presigned policy. Private folders return a key only.	Institution and LMS uploads depend on it
API CORS	Allow the production and staging origins on the API, not just localhost.	The live site cannot call the API today
Category images	Accept and store `imageUrl` on `POST` and `PUT /api/admin/categories`, and return it from `GET /api/categories`.	Admin category images are lost
Token refresh	`POST /api/auth/refresh` accepts a token that expired in the last 5 minutes and returns `{ token, user }`.	Users are signed out with no warning
Field names	Pick one name per pair and announce the removal date: `isEmailVerified` (not `isVerified`), `bio` (not `biography`), `socialLinks` (not `socials`), `avatarUrl` (not `avatar`). Category returns `{ _id, name, slug }` everywhere. Keep both spellings for 30 days.	The frontend handles both today; it should not have to
Staging	A staging API with seeded accounts for student, instructor, admin and institution admin, and a published OpenAPI 3 file at `/api/openapi.json` with a Postman collection.	The frontend cannot test signed-in routes without it
Health	Split `/api/health` into `/api/health/liveness` (process up) and `/api/health/readiness` (database and storage reachable).	The admin screen shows a fixed 99.9% uptime

The backend's own reply (13 October) commits to most of these within 48 to 72 hours, and staging within three business days.

Live data clean-up

The live site still shows test data from the Render database. The catalogue import does not remove it, so it must be deleted separately. Each item needs a backend admin or a database command, and should be recorded before it is removed.

- [ ] Unpublish or delete the course "Smoke Author Course 1792097872907", instructor "Jane Tutor", and any other course whose title starts with "Smoke" or "Test".
- [ ] Delete the test courses "Python Data Science Bootcamp" and "Fullstack JavaScript Mastery" (their thumbnails return 404).
- [ ] Remove the test accounts and enrolments that point to them.
- [ ] Confirm the live category list matches the 10 catalogue categories after the import.
- [ ] Run `npm run catalog:import` once on Render. It creates the 74 free courses and is safe to re-run.

Phase 1: partner applications

A prospective institution submits a request, Gloxad staff review it, and an approved request sends a single-use activation link. The review-first order was agreed in the backend's 13 October reply.

1. **Public request** `POST /api/institutions/applications`. Rate-limit to 5 per hour per IP, and consider CAPTCHA. Validate `institutionName`, `type`, `representative` (name, role, email) and optional fields with field-level errors (`{ error, fields }`). Create the application as `pending` with a reference ID such as `GLX-2026-9012`. Send a confirmation email. Reject a second pending application from the same representative email.
2. **Track** `GET /api/institutions/applications/:referenceId?code=` returns the status and submitted date only.
3. **Reviewer list and detail** `GET /api/admin/institution-applications` with search, filter by status and type, sort and paging. `GET /api/admin/institution-applications/:id` with documents as short-lived signed links.
4. **Decisions** `POST .../approve`, `.../reject`, `.../request-info`. Each writes a `ReviewEvent` (actor, action, from, to, time). Approval creates an institution in `pending_activation` and emails a single-use activation link valid for 7 days.
5. **Activation** `POST /api/institutions/activate` sets the password (minimum 12 characters) and signs the representative in.
6. **Audit** every decision and note is in `ReviewEvent`, shown in the reviewer drawer as "Last updated by [name] on [date]".

7. **Permissions:** a reviewer is an admin permission, not a separate login type. Staff management is needed later (no screen is designed yet).

Emails: confirmation of the request, a reviewer alert, the activation link, and a rejection or information request.

Phase 2: institution onboarding and billing

An approved institution completes a six-step wizard. Each step saves to the server, so it can be resumed on another device.

Step	Endpoints	Notes
1. Logo	`POST /api/upload/presigned-url` (folder `logos/`), `PUT /api/institutions/me/branding`	SVG, PNG, JPG or GIF, up to 2 MB, at most 800 by 800 pixels
2. Departments	`POST`, `PUT`, `DELETE /api/institutions/me/departments`	Name is required. Department code is unique per institution.
3. Tutors and students	`POST /api/institutions/me/members`, `POST .../members/import` (CSV), `GET` template	Member email is unique per institution. Auto-invite toggle and invite status.
4. Subscription	`GET /api/institutions/plans`, `POST .../subscription/checkout`	Premium or Enterprise; monthly or annual
5. Branding	`PUT /api/institutions/me/branding`	Display name, colour, tagline
6. Finish	Subscription payment activates the workspace and opens `/institution/dashboard`	Dashboard data: `GET /api/institutions/me/dashboard`

Payments. Use Paystack for NGN and USD, as agreed in the backend's 13 October reply. Recurring plans use Paystack Plans and Authorizations. Bank transfers use Paystack dedicated virtual accounts, and a manual receipt goes to a staff queue. Verify every webhook by signature, and make webhooks and checkout idempotent. Card details never reach our servers.

Roles. `institution_admin` manages only their own institution. Members are normal users with an `institutionId`. Every institution-scoped query must filter by `institutionId`, because the landing page promises data isolation.

Not yet agreed: how students are placed in cohorts and whether institutions get isolated cohorts (see the LMS section).

Phase 3: trimester LMS core

The LMS is under `/api/lms`. Students see only cohorts they are enrolled in, instructors only the cohorts they teach, and admins everything. Anything else returns 403.

Access. `GET /api/lms/me` returns `{ role, cohorts[{ _id, name, trimester, programTitle, currentWeek }], activeCohortId }`. The LMS unlocks through an active enrolment in a cohort, not by role alone.

Progress rules (proposed; confirm with the business).

- Overall progress = completed required items (lessons, quizzes, assignments) divided by all required items.
- Week progress = the same, for items due that week.
- Attendance = sessions attended divided by sessions held, excluding excused sessions.
- The design's figures (65% overall, 5 of 8 assignments, 100% attendance) must come from these rules, not fixed values.

Endpoints. `GET /api/lms/dashboard/student`, `GET /api/lms/cohorts/:id/curriculum`, `GET /api/lms/lessons/:id` (video behind a signed or token-gated URL), `PUT /api/lms/lessons/:id/progress` (idempotent), and the threaded lesson comments.

Quizzes. Pass mark 70%; time limit set on the quiz (15 minutes in the design) and enforced by the server through `expiresAt`. Correct answers are revealed only after a check or submit. One in-progress attempt per user. Start, save each answer, and submit: `POST /api/lms/quizzes/:id/attempts`, `POST /api/lms/quiz-attempts/:id/answers`, `POST /api/lms/quiz-attempts/:id/submit`. An expired attempt is submitted automatically.

Assignments. Submit a file (50 MB; zip, pdf or source) or a GitHub repository URL. Grading queue for instructors: `GET /api/lms/instructor/grading-queue`. Grading: `PUT /api/lms/submissions/:id/grade` with score, feedback and rubric scores, and it notifies the student.

Decisions still open: quiz retakes, late-submission and resubmission policy.

Phase 4: community, certificates and analytics

Community. Each cohort has four channels: General, Q&A, Project Showcase and Study Groups. `GET /api/lms/channels`, `GET` and `POST /api/lms/posts` (title up to 120 characters, body up to 5,000, sanitised, rate-limited), likes as a toggle, and replies.

Authors, instructors and admins can delete or moderate posts. Report a post with `POST /api/lms/posts/:id/report`.

Certificates. Issue automatically when a course is complete: required lessons done, quizzes passed and assignments graded. Each certificate has a public, unguessable `certificateId` and a verification page. Generate the PDF on the server with a QR code to that page. `GET /api/lms/certificates/me`, `GET /api/lms/certificates/:id/pdf` (signed link). Admins can revoke or reissue. The LinkedIn share needs the title, Gloxad Academy as the organisation, the issue month and year, and the credential ID and URL.

Instructor overview. `GET /api/lms/instructor/cohorts/:id/overview` (average completion, active students, average grade, module completion, grading queue) and the paged roster with attendance and progress.

Admin analytics. `GET /api/lms/admin/analytics` returns revenue (with its currency), enrolments with change, completion rate, platform health (uptime from the monitor, see Phase 0), and enrolment trends. `GET /api/lms/admin/certificates/recent`. The user table needs search, role filter, suspend and activate, and `lastLoginAt`. Show "Showing 1 to N of total" from the pagination total.

Acceptance: a completed course issues a verifiable certificate, and instructor and admin numbers match the database.

Phase 5: live classes

This is the largest single piece of work in the LMS, and the provider decision comes first.

Conflict to resolve. The backend's 13 October reply chose LiveKit Cloud. The backend's latest commit (`ad9456b`, "Live classes: stream through YouTube instead of LiveKit") moved live classes to YouTube. The team must pick one before building. Options and what each needs:

Option	Strengths	Gaps
LiveKit Cloud	Two-way video, rooms, recording, open-source React components	Cost per minute; latency from Nigeria to test
YouTube live	Free, reaches large audiences	One-way; no attendance or per-student join control; no private rooms
Daily, Agora or Zoom SDK	Managed rooms, alternatives to test	Cost and contracts to compare

Once a provider is chosen:

- `GET /api/lms/live-sessions` (with the next session for the countdown), `POST`, `PUT` and `DELETE` for instructors.
- `POST /api/lms/live-sessions/:id/join` returns a short-lived room token, only for enrolled users, and records attendance.
- Polls: create, vote once per user, results shared when the poll closes.
- Attendance: `GET` roster and instructor updates (present, absent, excused).
- Real time on the `/live` socket namespace, with the JWT in `auth.token`: `chat:message`, `hand:raise`, `hand:lower`, `poll:launched`, `poll:closed`, `participants:update`, `session:ended`. Persist chat so late joiners can read it.

Acceptance: a scheduled class goes live, students join with video, polls and chat work, and attendance appears in the roster.

Free courses and YouTube

All 74 catalogue courses are free. The backend changes below are in the local repository (`gloxard-backend`) and are not yet committed or deployed.

Import. `npm run catalog:import` creates or updates the 74 courses from `src/data/gloxadCourses2026.json`. A course with price 0 is imported as `courseType: "free"` and `enrollmentOpen: true`. Re-running it creates no duplicates and keeps any price an admin has set.

Enrolment. Free enrolment (`POST /api/enrollments/enroll/:courseId`) must work for every imported course. Paid checkout remains for future paid courses.

Video rules from the team's embedding guidelines (26 October):

- Free access only. No dummy price, paywall or subscription on any YouTube resource.
- Use the original YouTube embed, not downloads or copies. Show the creator, the original title and a direct link.
- Do not gate viewing (no like, subscribe or comment first) and do not charge to watch.
- Do not use free videos as a funnel to sell paid courses.
- Keep a content register for each video: title, creator, exact URL, date checked, embedding allowed and tested, attribution, approver and last review date.

Gaps the backend can help with: store the creator, the source URL and an `embedAllowed` flag per lesson, so the register and the course pages can be generated rather than kept by hand. The site currently checks that videos exist and are public, not that embedding is switched on for each one.

Cross-cutting requirements

Roles. `student` (own records, enrolled cohorts only). `instructor` (approved; cohorts they teach). `admin` (everything). Partner reviewer (a permission on admin). `institution_admin` and institution members (their own institution only).

Security and privacy.

- Validate everything on the server. The browser's checks are for convenience only.
- Rate-limit sign-in and password reset (for example 5 per minute per IP), partner applications, invitations, forum posts and uploads.
- Enforce file type and size through the presigned policy. Scan submissions and documents for malware. Serve private files only through signed links.
- Verify provider webhooks by signature. Keep the JWT secret in the environment. Bump `tokenVersion` on password change and sign-out everywhere. Never accept demo tokens in production.
- Keep an audit log of admin and reviewer actions. Logs must never contain tokens, passwords or card details.
- Agree retention, export and deletion for personal data, and publish the Terms and Privacy pages the footer already links to (they do not exist yet).

Email. One transactional provider. The sender must be verified with that provider. Templates for everything above, with plain-text versions. Bounce and failure handling. Deliverability (SPF, DKIM, DMARC) on the sending domain. The website's email templates are done (logo header, welcome, verification, password reset).

Conventions. Lists use `?page=&limit=` with `{ items, pagination }`. Errors use `{ error, fields }`. Indexes on every filter field. Idempotency for webhooks, approvals, invitations and certificate issuance. Soft-delete instead of hard delete where someone may need to recover or audit it.

Documentation and monitoring. Publish OpenAPI 3 and a Postman collection. Add liveness and readiness health checks, request IDs in logs, error tracking, and database backups with a tested restore.

Environment variables for the website: `BACKEND_URL`, `NEXT_PUBLIC_BACKEND_URL` (socket.io, needs WebSocket upgrades), `CORS_ORIGINS` on the API, and provider keys for email, payments and video.

Decisions the business must make

The backend cannot be finished until these are answered. Each one is listed with the phase it blocks.

- [ ] **Live-class provider** (LiveKit, YouTube, or another): blocks Phase 5.
- [ ] **Video hosting for lessons** (S3 with a CDN and signed links, or a video service such as Mux, Cloudflare Stream or Vimeo): blocks Phase 3.
- [ ] **Cohorts and pricing:** how students are placed in cohorts, trimester fees in naira and dollars, and whether institutions get their own cohorts: blocks Phase 2 and Phase 3.
- [ ] **Quiz retakes and late work:** attempts allowed, late-submission and resubmission rules: blocks Phase 3.
- [ ] **Progress rules:** confirm the proposal in the Phase 3 section: blocks Phase 3.
- [ ] **Catalogue access:** the live test report recommends letting visitors browse courses without an account. Earlier, you asked for sign-in on the course list. Which applies: blocks nothing on the backend, but changes the access rules.
- [ ] **Certificate fees:** the catalogue still lists certificate fee stages (₦50,000 to ₦400,000). The free-course rules say free content must not be used to sell paid courses. Keep, remove, or move them to a separate certificate page.
- [ ] **Registered company name and social profile links:** needed for the footer.
- [ ] **Partner request form:** who owns its copy and fields (the backend's 13 October reply says the frontend owns the copy and the backend owns validation).
- [ ] **Naira payouts for tutors:** the bank and payment provider for tutor withdrawals.

Acceptance checks and sources

Hand-over checklist (Phase 0):

- [ ] The media bucket exists, and a GET on a seeded image returns 200.
- [ ] A presigned PUT from `http://localhost:3000` and from the production site succeeds.
- [ ] An OPTIONS preflight from the production site returns the correct `Access-Control-Allow-*` headers.
- [ ] Staging answers with seeded users for each role, and an OpenAPI file is available.
- [ ] `POST /api/auth/refresh` returns a new token for a token that expired in the last 5 minutes.
- [ ] `POST /api/admin/categories` stores `imageUrl`, and `GET /api/categories` returns it.

Phase checks are in the delivery table of the requirements document (Part E): each phase is done when its listed outcome can be demonstrated end to end, not when the endpoint exists.

Sources

- Gloxad Platform: Backend Requirements, 7 October 2026 (`docs/Gloxad-Backend-Requirements.pdf`).
- Backend Response and Frontend Alignment, October 2026 (`BACKEND_TO_FRONTEND_ALIGNMENT_DOC.pdf`).
- Gloxad Academy LMS Recommendations (`Gloxad LMS Recommendations.pdf`).
- Gloxad Academy LMS Test Report: Tutor and Student Dashboards, 25 October 2026.
- Gloxad Academy YouTube Course Embedding Team Guidelines, 26 October 2026.
- Gloxad Academy Complete Course List 2026.